

Métricas: IA vs Desenvolvedor Sênior — Full-Stack SAP Fiori

Projeto analisado: Monitor de Execução em Massa SPED Backend: Pacote ABAP `/TAX/SPED_MONITOR` — Sistema ED2
(`awsntwpstx01.engdb.infra:8000`) Frontend: Aplicação SAPUI5/Fiori — `d:\ENGDB\monitor_exec_massa` Data da análise: 02/04/2026
Ferramenta IA: Cursor IDE + MCP SAP ADT (Claude)

1. Inventário de Objetos — Backend ABAP

O pacote `/TAX/SPED_MONITOR` implementa um monitor de execução em massa de obrigações fiscais SPED (ECD, ECF, EFD, PIS/COFINS) com gateway OData para frontend Fiori.

#	Objeto	Tipo	Linhas	Bytes	Descrição Funcional
1	<code>/TAX/CL_GW_MON_EXEC_DPC_EXT</code>	Classe	1.304	40.615	Gateway OData — Data Provider (lógica customizada)
2	<code>/TAX/CL_MON_EXEC_CTR</code>	Classe	928	30.449	Controller — orquestração de negócio
3	<code>/TAX/CL_GW_MON_EXEC_MPC</code>	Classe	909	50.090	Gateway OData — Model Provider (gerada)
4	<code>/TAX/CL_GW_MON_EXEC_DPC</code>	Classe	789	30.699	Gateway OData — Data Provider (gerada)
5	<code>/TAX/EFD_REPORT</code>	Programa	738	26.439	Report SPED EFD ICMS/IPI
6	<code>/TAX/EXEC_REPORT_JOB</code>	Programa	695	21.191	Execução de reports via JOB em background
7	<code>/TAX/ECD_REPORT</code>	Programa	684	26.956	Report SPED ECD
8	<code>/TAX/ECF_REPORT</code>	Programa	539	20.989	Report SPED ECF
9	<code>/TAX/PCO_REPORT</code>	Programa	539	21.386	Report SPED EFD Contribuições (PIS/COFINS)
10	<code>/TAX/CL_MON_EXEC_DAO</code>	Classe	324	8.911	Data Access Object — acesso a banco de dados
11	<code>/TAX/CX_MONITOR_EXEC</code>	Classe	77	2.107	Classe de exceção customizada
12	<code>/TAX/CL_GW_MON_EXEC_MPC_EXT</code>	Classe	27	655	Gateway OData — Model Provider (extensão vazia)
13	<code>/TAX/FG_MON_LIST</code>	Grupo Funções	21	1.290	Function group para manutenção de tabela
14	<code>/TAX/D_MON_EXEC</code>	Tabela	—	—	Tabela de execuções do monitor
15	<code>/TAX/D_MON_LISTA</code>	Tabela	—	—	Tabela de lista de execuções
16	<code>/TAX/ST_MONITOR_EXEC_MASSA</code>	Estrutura	—	—	Estrutura para execução em massa
17	<code>/TAX/ST_MONITOR_LISTA_EXEC</code>	Estrutura	—	—	Estrutura para lista de execuções
18	<code>/TAX/DE_MONITOR_ID</code>	Elemento Dados	—	—	Elemento de dados — ID do monitor
19	<code>/TAX/MONITOR_EXEC</code>	Classe Mensagens	—	—	Mensagens de erro/informação
20	<code>/TAX/GW_MONITOR_EXEC_MASSA</code>	Projeto OData	—	—	Projeto Gateway OData (SEGW)
21	<code>/TAX/ECF</code>	Transação	—	—	Transação de execução ECF
22	<code>/TAX/EXEC_REPORT</code>	Transação	—	—	Transação de execução de reports

Totais Backend:

Métrica	Valor
Total de objetos	22
Objetos com código-fonte	12 (5 programas + 7 classes)
Total de linhas de código (LOC)	7.574
Total de bytes	281.777 (~275 KB)
Objetos DDIC (tabelas, estruturas, elementos)	5
Objetos de configuração (transações, mensagens, projeto GW)	5

2. Inventário de Objetos — Frontend SAPUI5/Fiori

Aplicação **SAPUI5 TypeScript** gerada via SAP Fiori Tools (`@sap/generator-fiori:basic`), conectada ao serviço OData `GW_MONITOR_EXEC_MASSA_SRV` . Implementa a interface do usuário Fiori para o monitor de execução em massa.

Stack tecnológico: SAPUI5 1.138.0 | TypeScript 5.x | OData v2 | SAP Fiori Launchpad

2.1 Arquivos de Código-Fonte (excluindo testes)

#	Arquivo	Tipo	Linhas	Bytes	Descrição Funcional
1	<code>controller/Main.controller.ts</code>	Controller TS	2.787	104.806	Controller principal — toda a lógica de UI
2	<code>components/FilterBar/EngFilterBar.ts</code>	Componente TS	875	26.549	Componente customizado de FilterBar com variantes
3	<code>view/Main.view.xml</code>	View XML	166	9.107	View principal com tabela, toolbar e filtros
4	<code>manifest.json</code>	Config JSON	122	2.814	Manifest do app (routing, OData, i18n)
5	<code>view/fragments/ViewLogDialog.fragment.xml</code>	Fragment XML	108	4.972	Diálogo de visualização de log de execução
6	<code>view/fragments/PasteDataDialog.fragment.xml</code>	Fragment XML	84	3.915	Diálogo para colar dados de planilha
7	<code>view/fragments/NovaExecucao.fragment.xml</code>	Fragment XML	78	3.448	Diálogo de nova execução em massa
8	<code>view/fragments/ListaExecucao.fragment.xml</code>	Fragment XML	75	3.484	Diálogo de lista de execuções
9	<code>components/FilterBar/models/EngFilterBarModel.ts</code>	Model TS	66	1.858	Modelo de dados para variantes do FilterBar
10	<code>view/fragments/ItemExecucao.fragment.xml</code>	Fragment XML	63	3.286	Diálogo de item de execução individual
11	<code>localService/metadata.xml</code>	OData Metadata	54	7.228	Metadata OData para mock server
12	<code>components/FilterBar/EngFilterBar.gen.d.ts</code>	TypeScript Def	46	1.797	Definições de tipo geradas do FilterBar
13	<code>components/FilterBar/entities/FilterItem.ts</code>	Entity TS	40	814	Interfaces e enums para FilterItem
14	<code>index.html</code>	HTML	37	1.257	Página de entrada do app
15	<code>Component.ts</code>	Component TS	28	694	Componente raiz do app UI5
16	<code>model/models.ts</code>	Model TS	17	487	Factory de modelos (deviceModel)
17	<code>view/fragments/ReportValueHelp.fragment.xml</code>	Fragment XML	15	553	Value Help para seleção de report
18	<code>view/fragments/StatusValueHelp.fragment.xml</code>	Fragment XML	15	575	Value Help para seleção de status
19	<code>i18n/i18n.properties</code>	i18n	11	312	Textos de internacionalização
20	<code>components/FilterBar/entities/Variant.ts</code>	Entity TS	9	194	Interface para variantes de filtro
21	<code>view/App.view.xml</code>	View XML	8	250	View raiz (App container)
22	<code>css/style.css</code>	CSS	1	35	Estilos customizados
23	<code>.Ui5RepositoryTextFiles</code>	Config	1	7	Config de deploy BSP

2.2 Arquivos de Teste

#	Arquivo	Tipo	Linhas	Descrição
1	test/locate-reuse-libs.js	Util JS	236	Localizador de bibliotecas reutilizáveis
2	test/flpSandbox.js	Config JS	100	Configuração do FLP Sandbox
3	test/flpSandbox.html	HTML	82	Página do FLP Sandbox
4	test/integration/NavigationJourney.ts	OPA5 Test	32	Teste de navegação OPA5
5	test/integration/opaTests.qunit.html	HTML	30	Página de testes OPA5
6	test/unit/unitTests.qunit.html	HTML	28	Página de testes unitários
7	test/integration/pages/MainPage.ts	OPA5 Page	24	Page Object — MainPage
8	test/integration/pages/AppPage.ts	OPA5 Page	23	Page Object — AppPage
9	test/testsuite.qunit.ts	Config TS	15	Suite de testes QUnit
10	test/initFlpSandbox.js	Config JS	13	Inicialização do FLP Sandbox
11	test/testsuite.qunit.html	HTML	12	Página da suite de testes
12	test/integration/opaTests.qunit.ts	Config TS	11	Config de testes OPA5
13	test/unit/unitTests.qunit.ts	Config TS	10	Config de testes unitários
14	test/unit/controller/MainPage.controller.ts	Unit Test	10	Teste unitário do controller

2.3 Arquivos de Configuração do Projeto

Arquivo	Descrição
package.json	Dependências NPM e scripts (build, deploy, lint)
tsconfig.json	Configuração TypeScript
ui5.yaml	Configuração UI5 Tooling (produção)
ui5-local.yaml	Configuração UI5 Tooling (local com proxy)
ui5-mock.yaml	Configuração UI5 Tooling (mock server)
ui5-deploy.yaml	Configuração de deploy no BSP repository
.eslintrc	Regras ESLint
.gitignore	Arquivos ignorados pelo Git

Totais Frontend:

Métrica	Valor
Total de arquivos fonte	24 (excl. testes)
Total de arquivos teste	14
Linhas de código fonte (LOC)	4.718
Linhas de código teste	626
Total de bytes (fonte)	178.721 (~175 KB)
Frameworks	SAPUI5 1.138.0, TypeScript 5.x
Protocolo	OData v2

3. Totais Consolidados do Projeto Full-Stack

Métrica	Backend ABAP	Frontend UI5	Total
Objetos/Arquivos com código	12	24	36
Linhas de código (LOC)	7.574	4.718	12.292
Bytes de código	281 KB	175 KB	456 KB
Objetos de configuração/DDIC	10	8	18
Arquivos de teste	—	14	14

4. Análise de Complexidade — Backend ABAP

Objeto	LOC	Métodos	SELECTs	IFs	LOOPS	TRY/CATCH	RAISEs	CALLs	Complexidade
/TAX/CL_GW_MON_EXEC_DPC_EXT	1.304	15	4	40	12	4	1	9	Alta
/TAX/CL_MON_EXEC_CTR	928	25	0	34	3	2	4	6	Alta
/TAX/CL_GW_MON_EXEC_MPC	909	11	0	1	0	0	0	0	Média (gerada)
/TAX/CL_GW_MON_EXEC_DPC	789	33	0	12	0	0	11	2	Média (gerada)
/TAX/EFD_REPORT	738	—	0	52	2	16	0	0	Alta
/TAX/EXEC_REPORT_JOB	695	—	4	24	8	0	0	9	Alta
/TAX/ECD_REPORT	684	—	0	40	2	8	0	0	Alta
/TAX/ECF_REPORT	539	—	0	26	2	6	0	0	Média
/TAX/PCO_REPORT	539	—	0	28	2	8	0	0	Média
/TAX/CL_MON_EXEC_DAO	324	13	10	8	0	0	0	0	Média
/TAX/CX_MONITOR_EXEC	77	2	0	1	0	0	0	1	Baixa
/TAX/CL_GW_MON_EXEC_MPC_EXT	27	0	0	0	0	0	0	0	Baixa

Distribuição de complexidade backend: 5 objetos Alta | 5 objetos Média | 2 objetos Baixa

5. Análise de Complexidade — Frontend SAPUI5/Fiori

5.1 Código TypeScript

Arquivo	LOC	Métodos Pub	Métodos Priv	Event Handlers	IFs	Loops	Try/Catch	OData Ops	Complexidade
Main.controller.ts	2.787	49	34	56	178	—	22	5 (R/C/U/D)	Muito Alta
EngFilterBar.ts	875	6	25	8	30	9	—	—	Alta
EngFilterBarModel.ts	66	—	—	—	—	—	—	—	Baixa
FilterItem.ts	40	—	—	—	—	—	—	—	Baixa
Component.ts	28	—	—	—	—	—	—	—	Baixa
models.ts	17	—	—	—	—	—	—	—	Baixa
Variant.ts	9	—	—	—	—	—	—	—	Baixa

Destaques de complexidade — Main.controller.ts :

Indicador	Valor	Significado
Métodos públicos	49	Enorme superfície de API
Métodos privados	34	Lógica interna significativa
Event handlers (onXxx)	56	Alto acoplamento UI-lógica
Condicionais (if)	178	Complexidade ciclomática extrema
Blocos try/catch	22	Tratamento extensivo de erros
Callbacks OData (success/error)	22	Fluxos assíncronos complexos
MessageToast / MessageBox	31 / 22	UX rica com feedback ao usuário
Fragment.load (diálogos)	7	7 diálogos dinâmicos
new Filter()	17	Lógica de filtragem OData complexa
BusyIndicator	8	Gerenciamento de estado de carregamento

Destaques de complexidade — EngFilterBar.ts :

Indicador	Valor	Significado
Métodos (pub + priv)	31	Componente UI5 customizado robusto
Condicionais (if)	30	Lógica condicional significativa
switch/case	7 switches / 21 cases	Roteamento por tipo de controle
Loops (for/forEach/map)	9	Iteração sobre filtros e variantes
Eventos (attach/fire)	8	Comunicação via eventos UI5

5.2 Views e Fragments XML

Arquivo	Linhas	Controles UI	Event Bindings	Complexidade
Main.view.xml	166	2	9	Alta
ViewLogDialog.fragment.xml	108	3	1	Média
PasteDataDialog.fragment.xml	84	1	3	Média
NovaExecucao.fragment.xml	78	1	8	Média
ListaExecucao.fragment.xml	75	1	8	Média
ItemExecucao.fragment.xml	63	5	6	Média
ReportValueHelp.fragment.xml	15	1	0	Baixa
StatusValueHelp.fragment.xml	15	1	0	Baixa
App.view.xml	8	1	0	Baixa

Distribuição de complexidade frontend: 2 Muito Alta/Alta (TS) | 5 Média (XML) | 7 Baixa

6. Estimativa de Tempo — Desenvolvedor Sênior (Humano)

6.1 Backend ABAP — Desenvolvedor ABAP Sênior

Premissas para um desenvolvedor ABAP sênior (5+ anos de experiência):

- Produtividade média: **30-50 LOC/hora** para código customizado novo
- Produtividade em código gerado (SEGW): **100-150 LOC/hora** (assistido por ferramenta)
- Inclui: análise, codificação, testes unitários, ativação e documentação
- Não inclui: especificação funcional, testes integrados, aprovações

Objeto	LOC	Tipo Trabalho	Horas Dev Sênior	Justificativa
Objetos DDIC (tabelas, estruturas, elementos)	—	Configuração	4h	5 objetos × 0,8h (definição campos, domínios, elementos)
Classe de mensagens	—	Configuração	1h	Definição de mensagens de erro/info
Transações	—	Configuração	0,5h	2 transações SE93
Projeto OData (SEGW)	—	Geração	4h	Modelagem de entidades, geração, registro
/TAX/CL_GW_MON_EXEC_DPC	789	Gerado (SEGW)	2h	Código gerado pelo framework OData
/TAX/CL_GW_MON_EXEC_MPC	909	Gerado (SEGW)	2h	Código gerado pelo framework OData
/TAX/CL_GW_MON_EXEC_MPC_EXT	27	Extensão vazia	0,5h	Shell class, sem lógica
/TAX/CL_GW_MON_EXEC_DPC_EXT	1.304	Customizado	32h	Lógica OData complexa: 15 métodos, SELECTs, IFs, LOOPS
/TAX/CL_MON_EXEC_CTR	928	Customizado	24h	Controller com 25 métodos, orquestração de negócio
/TAX/CL_MON_EXEC_DAO	324	Customizado	8h	DAO com 13 métodos e 10 SELECTs
/TAX/CX_MONITOR_EXEC	77	Customizado	1h	Classe de exceção simples
/TAX/ECD_REPORT	684	Customizado	18h	Report SPED com lógica fiscal complexa
/TAX/ECF_REPORT	539	Customizado	14h	Report SPED com lógica fiscal
/TAX/EFD_REPORT	738	Customizado	20h	Report SPED com 52 IFs e 16 TRY/CATCHs
/TAX/PCO_REPORT	539	Customizado	14h	Report SPED PIS/COFINS
/TAX/EXEC_REPORT_JOB	695	Customizado	18h	Execução em batch com JOBS
/TAX/FG_MON_LIST	21	Manutenção	2h	Function group com gerador de manutenção

Subtotal Backend — Dev Sênior:

Fase	Horas
Objetos DDIC e configuração	9,5h
Código gerado (SEGW/OData)	8,5h
Código customizado (classes + reports)	149h
Testes unitários e integração	24h
Revisão de código e documentação	8h
Subtotal Backend	199h

6.2 Frontend SAPUI5 — Desenvolvedor Fiori/UI5 Sênior

Premissas para um desenvolvedor SAPUI5/Fiori sênior (3+ anos de experiência):

- Produtividade média: **25-40 LOC/hora** para lógica de controller TypeScript
- Produtividade em XML views/fragments: **40-60 LOC/hora**
- Produtividade em componentes customizados: **20-35 LOC/hora**
- Inclui: análise, codificação, testes, debug em navegador, ajustes de UX
- Não inclui: design UX/UI, especificação funcional

Arquivo	LOC	Tipo Trabalho	Horas Dev Sênior	Justificativa
Main.controller.ts	2.787	Customizado	80h	83 métodos, 178 IFs, 56 event handlers, 22 try/catch, 7 diálogos, OData CRUD. Controller monolítico de complexidade extrema
EngFilterBar.ts	875	Componente Custom	28h	Componente UI5 customizado: 31 métodos, 7 switches, gestão de variantes, integração FLP
EngFilterBarModel.ts	66	Customizado	2h	Modelo de variantes com persistência
FilterItem.ts + Variant.ts	49	Customizado	1,5h	Interfaces e enums (tipagem TypeScript)
EngFilterBar.gen.d.ts	46	Gerado	0,5h	Definição de tipos gerada
Main.view.xml	166	View XML	4h	View principal com toolbar, tabela, bindings
6 fragments XML	423	Fragments XML	10h	6 diálogos complexos (Nova Execução, Lista, Log, Paste Data, Value Helps)
App.view.xml	8	View XML	0,5h	Container shell
Component.ts + models.ts	45	Boilerplate	1h	Inicialização padrão UI5
manifest.json	122	Configuração	2h	Routing, OData datasource, i18n, dependências
localService/metadata.xml	54	OData Mock	2h	Metadata para mock server local
index.html + i18n + css	49	Config/Estilo	1h	Arquivos de suporte
Configuração projeto	—	Config	3h	package.json, tsconfig, ui5.yaml (x3), deploy, eslint
Testes OPA5 + QUnit	626	Testes	10h	14 arquivos de teste (integration + unit)

Subtotal Frontend — Dev Sênior:

Fase	Horas
Controller principal (Main.controller.ts)	80h
Componente customizado (EngFilterBar)	31,5h
Views XML e fragments	14,5h
Boilerplate e configuração	9,5h
Testes OPA5/QUnit	10h
Debug, ajustes de UX e integração OData	16h
Subtotal Frontend	161,5h

6.3 Resumo Total — Desenvolvedor Sênior (Humano)

Camada	Horas	% do Total
Backend ABAP	199h	55%
Frontend UI5	161,5h	45%
Total Desenvolvedor Sênior	360,5h	100%

~360,5 horas ≈ 45 dias úteis ≈ 9 semanas (considerando 8h/dia)

7. Estimativa de Tempo — IA (Cursor + MCP SAP ADT)

7.1 Backend ABAP — IA

Premissas para desenvolvimento assistido por IA:

- IA gera código ABAP a partir de especificação em linguagem natural
- Desenvolvedor humano (revisor) supervisiona, valida e ajusta
- IA faz deploy direto no SAP via MCP (`sap_write_source` , `sap_activate`)
- Produtividade de geração: **150-300 LOC/hora** (incluindo iterações de correção)
- Revisão humana: **20-30%** do tempo original para validação

Objeto	LOC	Horas IA + Revisão	Detalhamento
Objetos DDIC	—	3h	IA gera definições; humano valida domínios/campos (economia: 30%)
Classe de mensagens	—	0,3h	IA gera em segundos; validação rápida
Transações	—	0,3h	Criação automatizada via ADT
Projeto OData (SEGW)	—	3h	Modelagem requer decisão humana; IA auxilia
<code>/TAX/CL_GW_MON_EXEC_DPC</code>	789	1h	Gerado (SEGW) + revisão
<code>/TAX/CL_GW_MON_EXEC_MPC</code>	909	1h	Gerado (SEGW) + revisão
<code>/TAX/CL_GW_MON_EXEC_MPC_EXT</code>	27	0,2h	Shell trivial
<code>/TAX/CL_GW_MON_EXEC_DPC_EXT</code>	1.304	10h	IA gera bulk; humano valida lógica OData (iterações)
<code>/TAX/CL_MON_EXEC_CTR</code>	928	8h	IA gera orquestração; humano valida regras de negócio
<code>/TAX/CL_MON_EXEC_DAO</code>	324	2h	SELECTs e DAO são bem padronizados para IA
<code>/TAX/CX_MONITOR_EXEC</code>	77	0,3h	Classe de exceção — IA gera em segundos
<code>/TAX/ECD_REPORT</code>	684	6h	IA gera estrutura; humano valida lógica fiscal SPED
<code>/TAX/ECF_REPORT</code>	539	5h	Similar ao ECD com ajustes
<code>/TAX/efd_REPORT</code>	738	7h	Mais complexo (52 IFs); revisão fiscal detalhada
<code>/TAX/PCO_REPORT</code>	539	5h	Similar ao ECF
<code>/TAX/EXEC_REPORT_JOB</code>	695	5h	Batch processing — padrão bem conhecido pela IA
<code>/TAX/FG_MON_LIST</code>	21	0,5h	Function group simples

Subtotal Backend — IA:

Fase	Horas
Objetos DDIC e configuração	6,6h
Código gerado (SEGW/OData)	5,2h
Código customizado (classes + reports)	48,3h
Revisão humana e validação funcional	8h
Testes e iterações de correção com IA	6h
Subtotal Backend	74,1h

7.2 Frontend SAPUI5 — IA

Premissas para desenvolvimento frontend assistido por IA:

- IA gera código TypeScript/XML UI5 a partir de descrição funcional
- IA conhece bem padrões SAPUI5, OData v2, e Fiori Design Guidelines
- Produtividade de geração: **100-200 LOC/hora** para controllers TS
- Produtividade em XML: **200-400 LOC/hora** (alta padronização)
- Revisão humana para UX e integração: **25-35%** do tempo original

Arquivo	LOC	Horas IA + Revisão	Detalhamento
Main.controller.ts	2.787	28h	IA gera métodos CRUD/OData e event handlers; humano revisa lógica de 178 IFs, integração diálogos e fluxos complexos. Economia alta pela padronização OData
EngFilterBar.ts	875	10h	Componente customizado: IA gera estrutura e switch/case; humano ajusta integração com FLP e gestão de variantes
EngFilterBarModel.ts	66	0,5h	Modelo simples — IA gera rapidamente
FilterItem.ts + Variant.ts	49	0,3h	Interfaces TypeScript — IA gera instantaneamente
EngFilterBar.gen.d.ts	46	0,2h	Geração de tipos automática
Main.view.xml	166	1,5h	IA gera XML views UI5 com alta qualidade
6 fragments XML	423	3h	IA gera fragments de diálogos rapidamente; humano ajusta bindings
App.view.xml	8	0,1h	Boilerplate trivial
Component.ts + models.ts	45	0,2h	Boilerplate UI5 — IA gera em segundos
manifest.json	122	1h	IA gera config; humano valida routing e datasources
localService/metadata.xml	54	0,5h	IA gera a partir do modelo OData
index.html + i18n + css	49	0,3h	Arquivos de suporte triviais
Configuração projeto	—	1h	IA gera configs; humano valida dependências
Testes OPA5 + QUnit	626	3h	IA gera testes a partir dos controllers
Debug e ajustes de UX	—	5h	Validação visual no browser e ajustes de UX

Subtotal Frontend — IA:

Fase	Horas
Controller principal (Main.controller.ts)	28h
Componente customizado (EngFilterBar)	11h
Views XML e fragments	4,6h
Boilerplate e configuração	3,3h
Testes OPA5/QUnit	3h
Debug, ajustes de UX e integração OData	5h
Subtotal Frontend	54,9h

7.3 Resumo Total — IA (Cursor + MCP)

Camada	Horas	% do Total
Backend ABAP	74,1h	57%
Frontend UI5	54,9h	43%
Total IA + Revisão Humana	129h	100%

~129 horas ≈ 16,1 dias úteis ≈ ~3,2 semanas (considerando 8h/dia)

8. Comparativo Consolidado Full-Stack

8.1 Comparativo por Camada

Camada	Dev Sênior	IA + Revisão	Economia (h)	Economia %
Backend ABAP	199h	74,1h	124,9h	63%
Frontend UI5	161,5h	54,9h	106,6h	66%
Total Full-Stack	360,5h	129h	231,5h	64%

8.2 Comparativo Consolidado

Indicador	Dev Sênior	IA + Revisão	Economia	Economia %
Tempo total (horas)	360,5h	129h	231,5h	64%
Dias úteis (8h/dia)	45 dias	16,1 dias	28,9 dias	64%
Semanas	9 semanas	~3,2 semanas	~5,8 semanas	64%
Custo estimado*	R\$ 72.100	R\$ 25.800 + R\$ 4.990**	R\$ 41.310	57%

* Custo hora desenvolvedor sênior (ABAP/Fiori): R\$ 200/h (PJ mercado BR 2026) ** Custo licença Cursor Pro (~R\$ 100/mês × 3,2 sem ≈ R\$ 80) + API IA (~R\$ 4.910 para ~129h de uso intensivo)

8.3 Comparativo Apenas Backend (referência anterior)

Indicador	Dev Sênior	IA + Revisão	Economia	Economia %
Tempo total (horas)	199h	74,1h	124,9h	63%
Custo estimado	R\$ 39.800	R\$ 17.680	R\$ 22.120	56%

9. Indicadores de ROI e Viabilidade

9.1 Indicadores Quantitativos

Indicador	Só Backend	Full-Stack	Benchmark	Avaliação
Redução de tempo	63%	64%	>40% é excelente	✅ Excelente
Economia por projeto	R\$ 22.120	R\$ 41.310	—	✅ Muito significativa
ROI por projeto	12,4x	8,3x	>3x é positivo	✅ Muito alto
Payback	1 projeto	1 projeto	<3 projetos	✅ Imediato
LOC/hora (Dev Backend)	~38	—	30-50 (padrão ABAP)	Dentro da média
LOC/hora (Dev Frontend)	—	~29	25-40 (padrão UI5)	Dentro da média
LOC/hora (IA Backend)	~102	—	—	2,7x mais rápido
LOC/hora (IA Frontend)	—	~86	—	3,0x mais rápido
LOC/hora (IA Full-Stack)	—	~95	—	2,8x mais rápido
Taxa aceleração Backend	2,7x	—	>2x é bom	✅ Acima do esperado
Taxa aceleração Frontend	—	2,9x	>2x é bom	✅ Acima do esperado
Taxa aceleração Full-Stack	—	2,8x	>2x é bom	✅ Acima do esperado

9.2 Indicadores Qualitativos

Dimensão	Dev Sênior	IA + Revisão	Vantagem
Consistência de código	Variável (estilo pessoal)	Alta (padrões uniformes)	IA
Cobertura de edge cases	Baseada em experiência	Requer validação humana	Empate
Conhecimento de domínio fiscal	Alto (experiência acumulada)	Baixo (depende de contexto/prompt)	Dev
Documentação inline	Frequentemente omitida	Gera automaticamente	IA
Refatoração	Lenta (análise manual)	Rápida (análise automatizada)	IA
Testes unitários	Muitas vezes negligenciados	Pode gerar junto com o código	IA
Deploy e ativação ABAP	Manual (Eclipse ADT)	Automatizado (MCP)	IA
Deploy frontend BSP	Manual (UI5 CLI)	Assistido (IA gera configs)	IA
Risco de bugs lógicos	Baixo (experiência)	Médio (requer revisão)	Dev
UX/Design	Requer designer ou experiência	Gera baseline; humano refina	Empate
Cross-stack (ABAP ↔ UI5)	Raro ter dev full-stack SAP	IA transita entre camadas naturalmente	IA

9.3 Análise de Risco

Risco	Probabilidade	Impacto	Mitigação
IA gera código com bug lógico	Média	Alto	Revisão humana obrigatória + testes
IA não entende regra fiscal específica	Alta	Médio	Fornecer especificação detalhada como contexto
Dependência excessiva da IA	Baixa	Médio	Manter equipe capacitada em ABAP/Fiori
Custos de API IA crescem	Baixa	Baixo	Modelos locais como fallback futuro
Código gerado não passa no ATC	Baixa	Baixo	IA pode corrigir iterativamente
IA gera UI inconsistente com Fiori Guidelines	Média	Médio	Fornecer Fiori Design Guidelines como contexto
Problemas de performance frontend	Média	Médio	Testes de carga manuais e profiling no browser

10. Análise por Tipo de Tarefa — Onde a IA Mais Economiza

10.1 Backend ABAP

Tipo de Tarefa	Economia IA	Justificativa
Código boilerplate (DAO, exceções, shells)	80-90%	Padrões repetitivos; IA gera instantaneamente
Reports com lógica estruturada	60-70%	IA gera estrutura completa; humano ajusta regras
Lógica de negócio complexa (Controller)	50-60%	IA gera esqueleto; humano implementa regras específicas
Gateway OData (DPC_EXT)	60-70%	Padrões OData bem documentados; IA conhece bem
Objetos DDIC (tabelas, estruturas)	30-40%	Decisões de design requerem humano
Configuração (transações, mensagens)	50-60%	Automatização via MCP reduz tempo manual
Testes e depuração ABAP	40-50%	IA auxilia geração de testes; execução é manual

10.2 Frontend SAPUI5/Fiori

Tipo de Tarefa	Economia IA	Justificativa
XML Views e Fragments	80-85%	UI5 XML é altamente padronizado; IA gera com alta qualidade
Boilerplate UI5 (Component, models, configs)	85-90%	Código repetitivo; IA gera instantaneamente
Controller — event handlers padrão	70-80%	Padrões OData CRUD muito bem conhecidos pela IA
Controller — lógica de UI complexa	55-65%	IA gera base; humano revisa fluxos de 178+ IFs
Componentes UI5 customizados (EngFilterBar)	60-65%	IA gera estrutura; humano ajusta ciclo de vida UI5
Manifest, routing, configs	65-75%	IA gera; humano valida datasources e dependências
Testes OPA5/QUnit	60-70%	IA gera testes a partir do código; humano valida cenários
Debug e ajustes UX	30-40%	Requer inspeção visual no browser; IA auxilia com sugestões

10.3 Cross-Stack (Full-Stack)

Tipo de Tarefa	Economia IA	Justificativa
Integração OData (backend ↔ frontend)	65-75%	IA entende ambos os lados; gera entities/bindings consistentes
Resolução de bugs cross-stack	50-60%	IA analisa request/response OData e sugere correções em ambas as camadas
Prototipação full-stack	75-85%	IA gera backend + frontend de forma coerente a partir de uma especificação

11. Projeção Anual de Economia

Considerando uma equipe de desenvolvimento SAP full-stack com volume típico de projetos:

Cenário	Projetos/ano	Horas economizadas	Economia anual (R\$)
Conservador (3 projetos full-stack)	3	694h	R\$ 123.480
Moderado (6 projetos full-stack)	6	1.389h	R\$ 246.960
Agressivo (12 projetos variados)	12	2.778h	R\$ 493.920

Investimento anual em IA:

Item	Custo/mês	Custo/ano
Cursor Pro (por desenvolvedor)	R\$ 100	R\$ 1.200
API IA (uso intensivo)	R\$ 500	R\$ 6.000
Total por desenvolvedor	R\$ 600	R\$ 7.200

ROI anual (cenário moderado): $R\$ 246.960 / R\$ 7.200 = 34,3x$

11.1 Comparativo com análise apenas Backend

Cenário Moderado (6 proj/ano)	Só Backend	Full-Stack	Diferença
Horas economizadas	750h	1.389h	+85%
Economia anual	R\$ 132.240	R\$ 246.960	+87%
ROI anual	18,4x	34,3x	+86%

Incluir o frontend na análise quase **dobra a economia** projetada, demonstrando que o impacto da IA é ainda maior em projetos full-stack SAP Fiori.

12. Recomendação

Veredicto:  **CONTINUAR INVESTINDO EM IA + CURSOR**

Com base nos indicadores analisados para o projeto full-stack:

- Economia de tempo de 64%** na implementação full-stack — cada 9 semanas de trabalho humano são reduzidas para ~3,2 semanas com IA.
- 231,5 horas economizadas por projeto** — equivalente a quase 29 dias úteis por projeto full-stack.
- ROI de 34,3x no cenário moderado** — o investimento em licenças e API se paga já no primeiro projeto, com retorno exponencial em escala.
- IA é especialmente eficaz no frontend** (66% de economia) — XML views, controllers OData e boilerplate UI5 são altamente padronizados.
- IA como desenvolvedor full-stack SAP** — diferentemente de humanos (que raramente dominam ABAP + Fiori), a IA transita naturalmente entre backend e frontend, gerando código coerente em ambas as camadas.
- Revisão humana continua indispensável** — especialmente para regras fiscais SPED (backend) e UX/interação (frontend).
- MCP SAP ADT como diferencial** — a integração direta Cursor → SAP elimina o ciclo manual de copiar/colar código entre IDE e SAP GUI, economizando ~15% adicional no fluxo de deploy.

Próximos Passos Recomendados

Ação	Prioridade	Prazo
Expandir uso do Cursor + MCP para toda a equipe ABAP/Fiori	Alta	30 dias
Criar templates de prompts para padrões fiscais SPED	Alta	15 dias
Criar templates de prompts para padrões UI5/Fiori	Alta	15 dias
Documentar regras de negócio como contexto para IA	Média	60 dias
Medir tempo real em 3 projetos piloto full-stack	Alta	90 dias
Avaliar modelos de IA locais para reduzir custo de API	Baixa	180 dias
Criar componentes UI5 reutilizáveis com assistência IA	Média	90 dias

Nota metodológica: As estimativas de tempo foram calculadas com base em benchmarks da indústria de desenvolvimento ABAP (ISBSG, SAP Community, ASUG) e SAPUI5/Fiori (SAP Fiori Design Guidelines, UI5 Community), e na análise de complexidade ciclomática do código-fonte real do pacote ABAP `/TAX/SPED_MONITOR` e da aplicação frontend `monitor_exec_massa` (d:\ENGDB\monitor_exec_massa). Os tempos de IA foram estimados com base na experiência prática de geração de código ABAP e TypeScript/UI5 via Cursor IDE com MCP ADT durante o desenvolvimento deste próprio projeto.